

Machine Learning - Deep Learning fundamentals (69152)

Master in Robotics, Graphics and Computer Vision
Ana C. Murillo



Universidad
Zaragoza

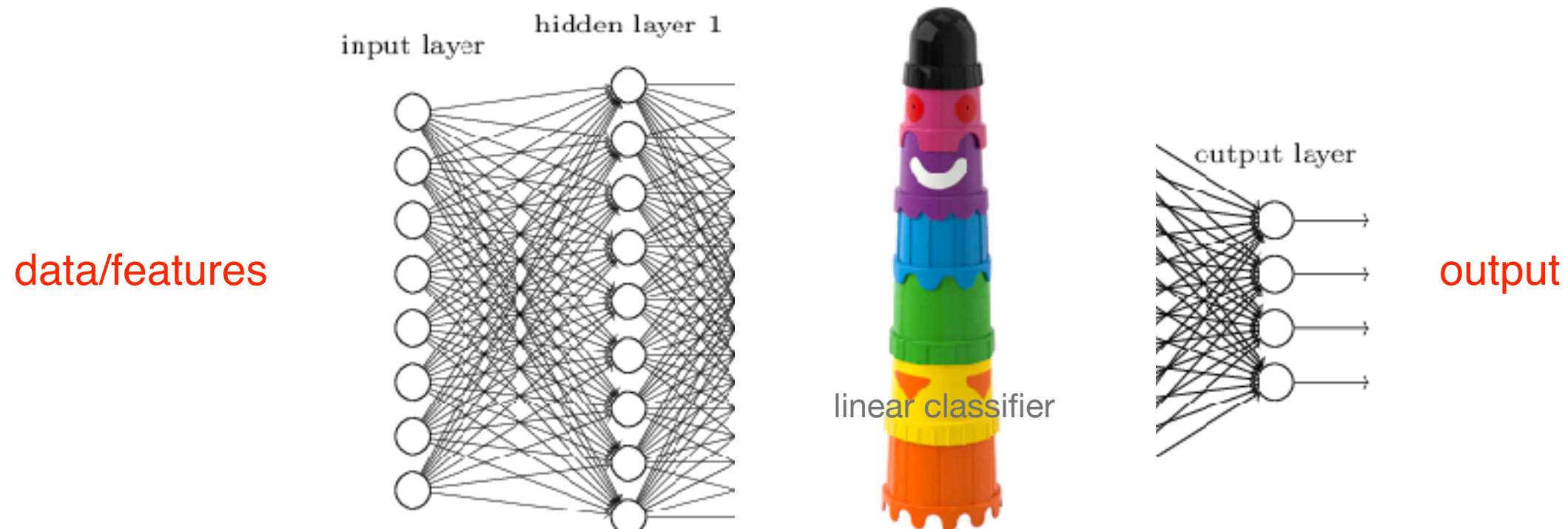
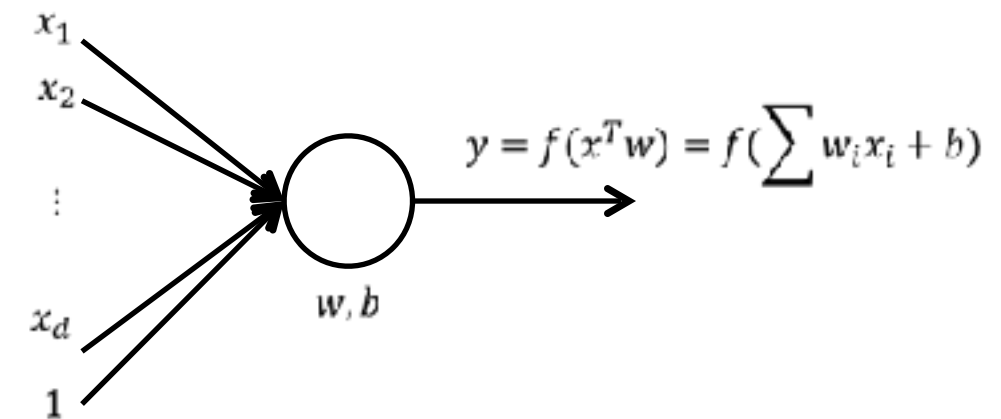
Next

- What's deep learning?
 - Related topics
 - DL pipeline
- Fundamentals of DL
 - Review basic concepts
 - **NN and DNN**

Neural Networks and Deep Neural Networks

- **Neural Networks**

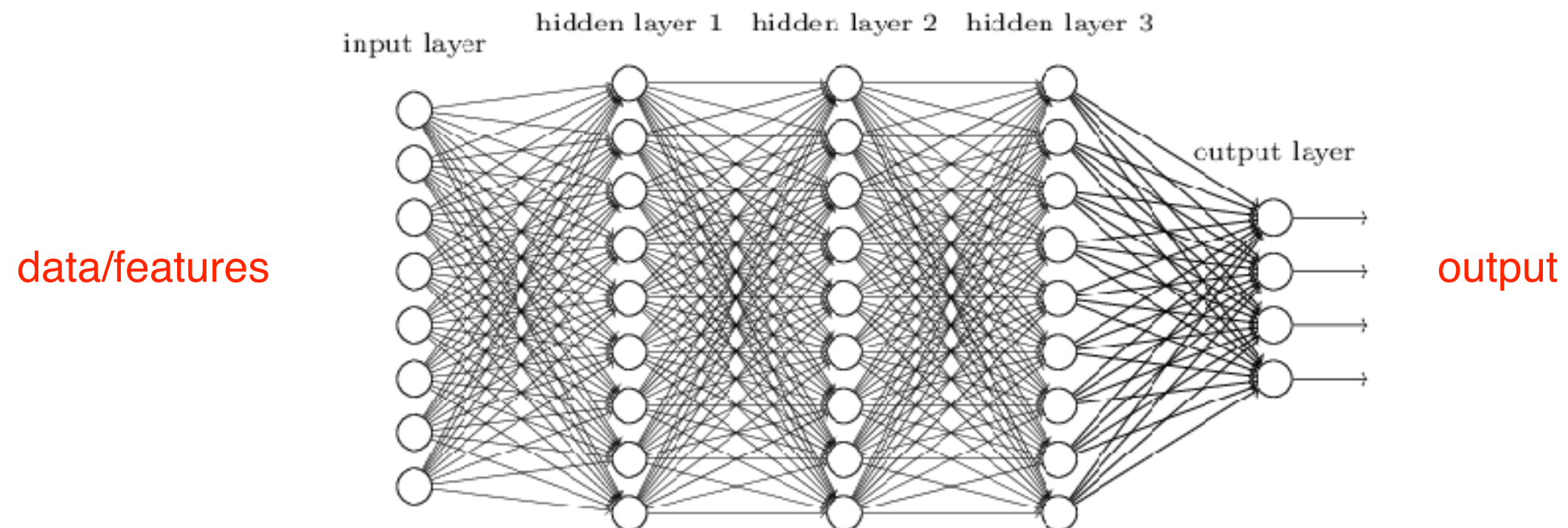
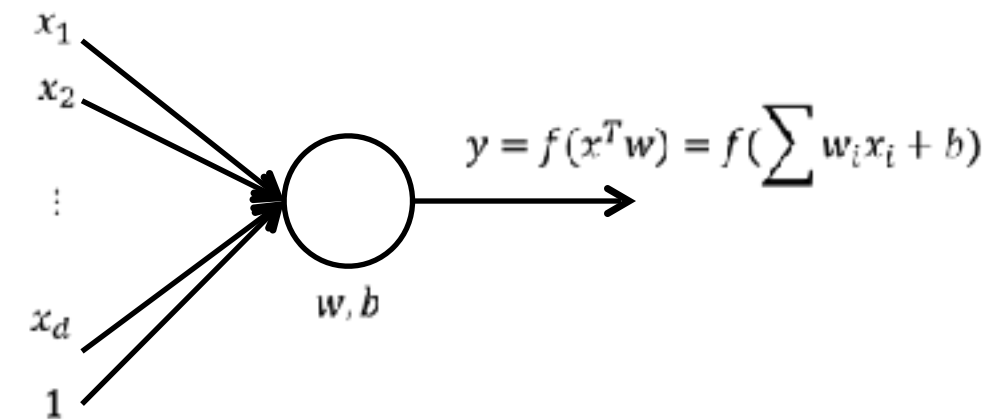
- **Neuron:** atomic computational unit in NN.
Params: w , b and f .
- Activation function (f) usually non linear
- Neural Network: connect several layers, neurons, ...
- *Perceptron*: simplest NN. 1 layer - binary linear classifier



Neural Networks and Deep Neural Networks

- **Neural Networks**

- Neuron: atomic computational unit in NN.
Params: w , b and f .
- Activation function (f) usually non linear
- Neural Network: connect several layers, neurons, ...
- **Deep** Neural Network: many hidden layers



Deep Neural Network

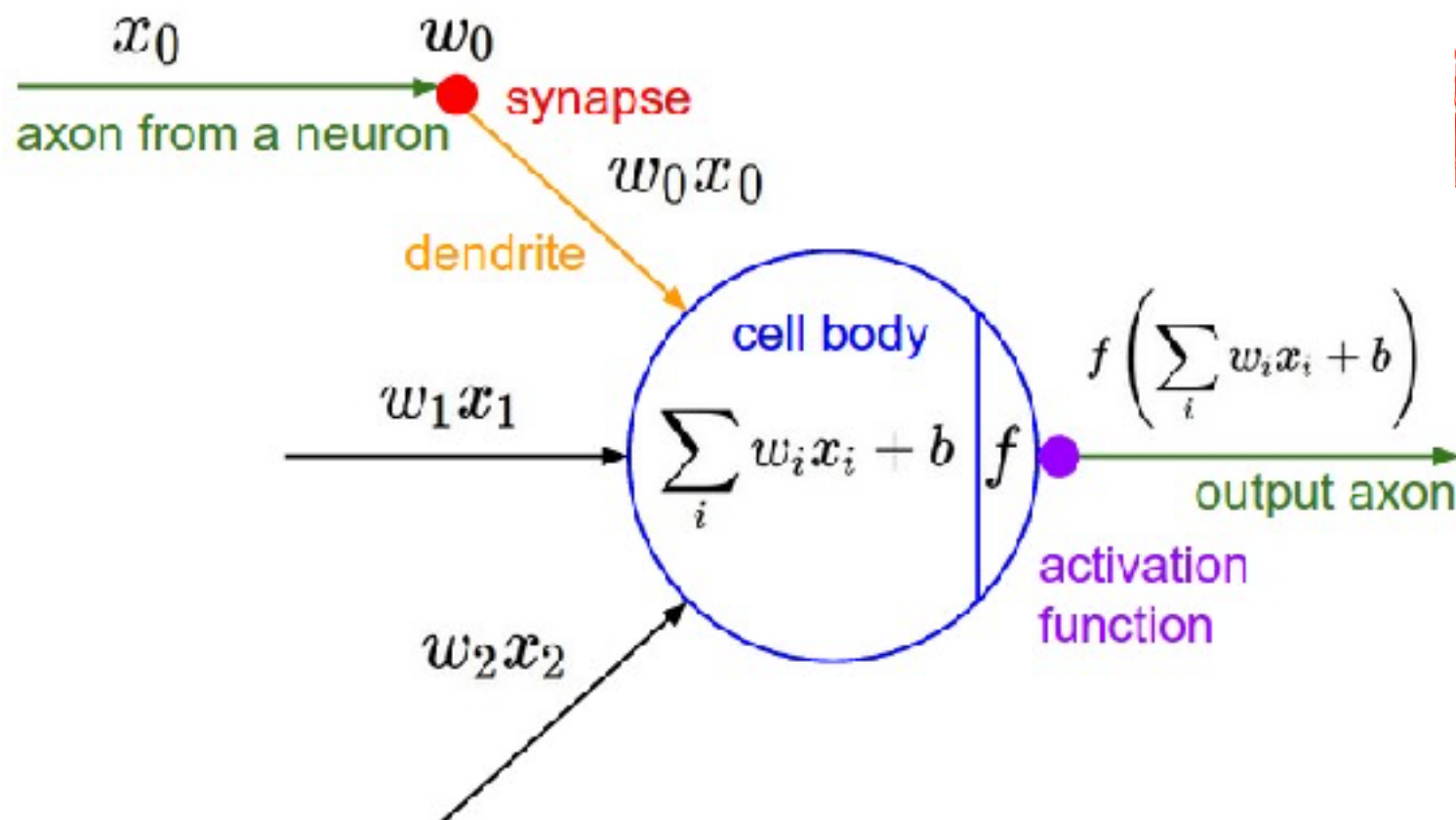
- Deep **feed-forward** Neural Networks:
data propagated through the network to predict the output
- **Some important DNN *ideas* and *ingredients***
 - forward-backward pass
 - activation, optimization, gradient descent, backpropagation
 - parameters (model) and hyperparameters (config.)

Deep Neural Network



Forward pass (prediction, inference, ...)

of a fully-connected layer: *one matrix multiplication followed by a bias offset and activation function*



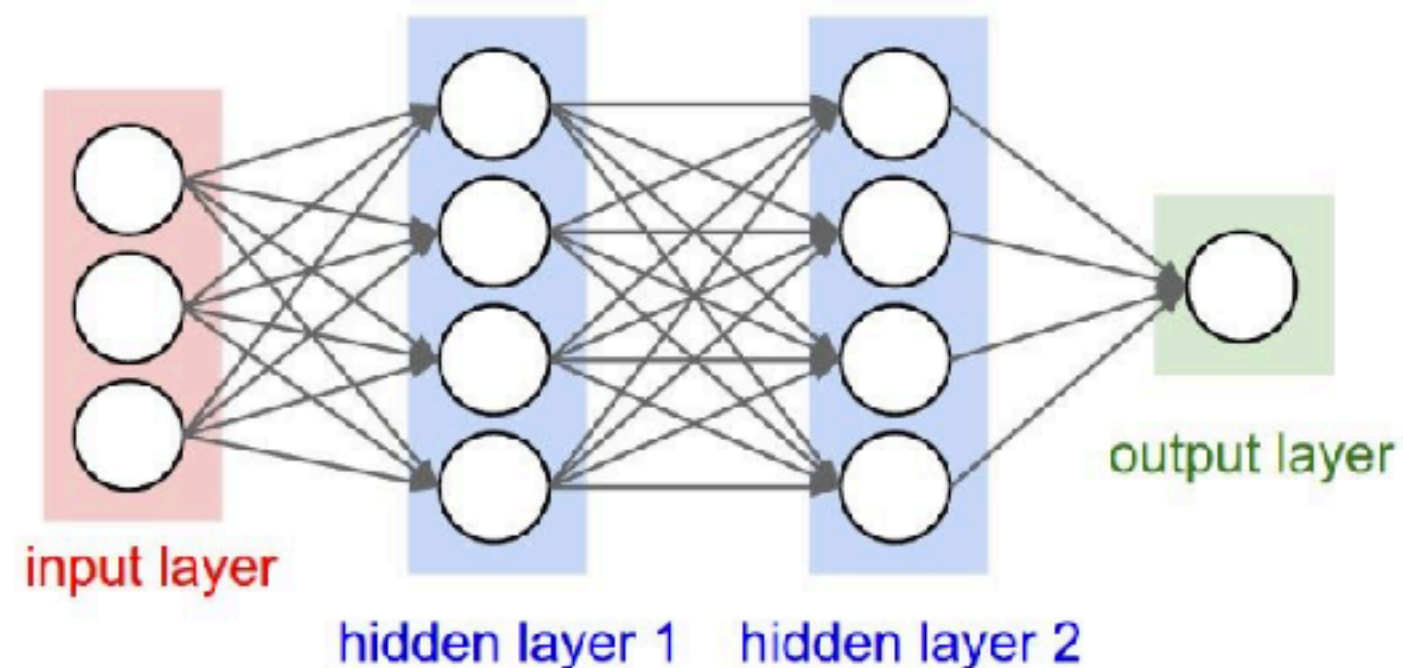
**What do we compute here?
Score, Loss, Optimization?**

Deep Neural Network



Forward pass. Feed-forward computation on a 3 layer NN

$$f(W1, W2, W3) = a(W1, b(W2, c(W3)))$$



```
fc1 = X.dot(W1) + b1      # A?  
X2 = f(fc1)               # B?  
fc2 = X2.dot(W2) + b2    # C?  
X3 = f(fc2)               # D?  
scores = X3.dot(W3) + b3 # E?
```

What's A, B, C, D, E?

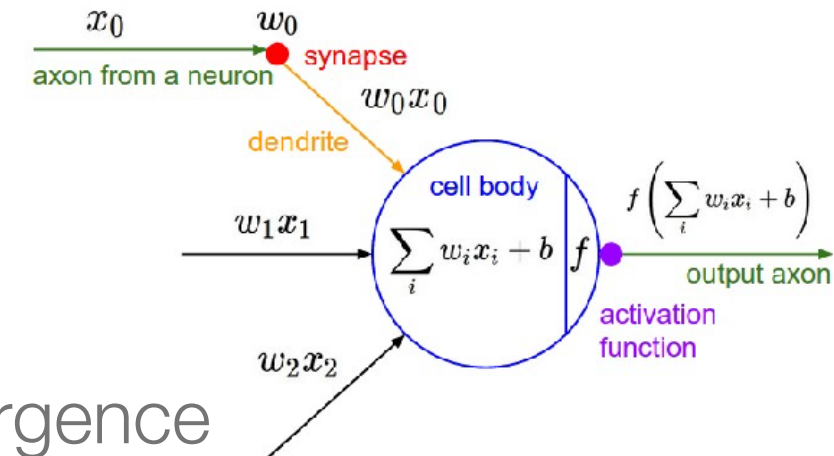
Deep Neural Network



- **Activation** of a Neuron - **non linearity!**

- ReLU very popular

$$f(x) = \max(0, x)$$

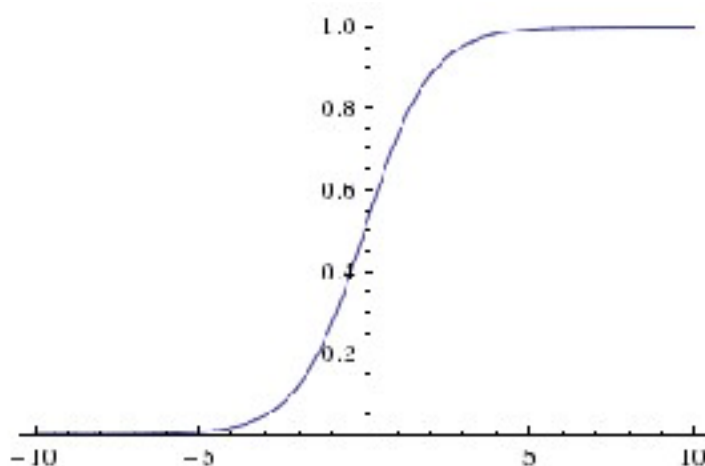


- fast to compute and shown to accelerate convergence

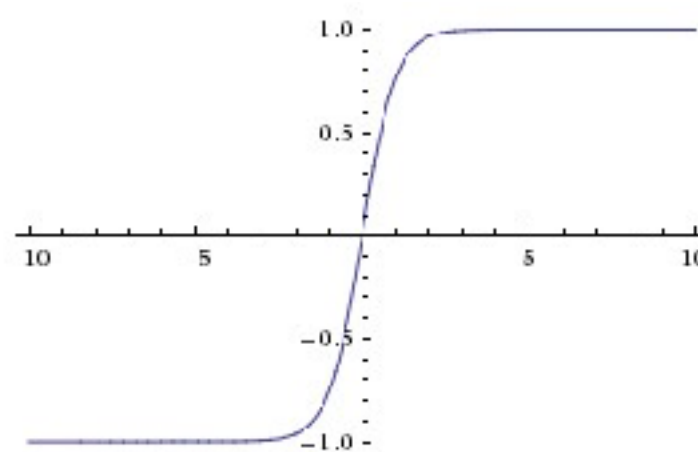
- weak. high learning rate can “kill” many of the neurons (never activated)

- generalisation $\rightarrow$ MaxOut unit $\max(w_1^T x + b_1, w_2^T x + b_2)$.

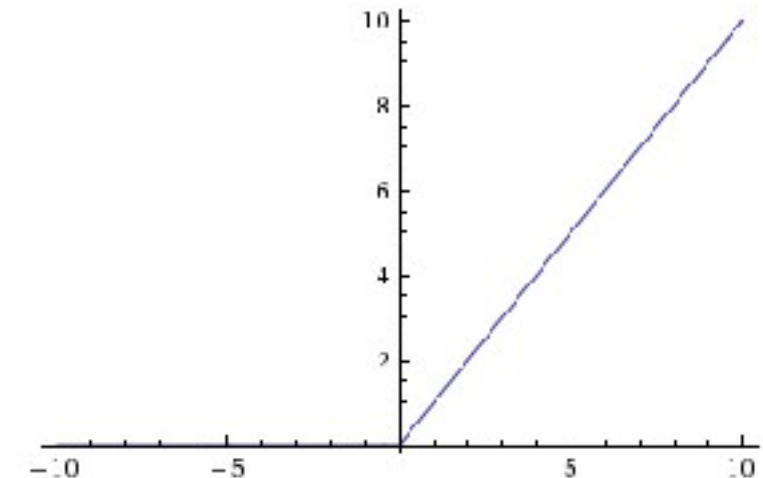
Sigmoid



Tanh



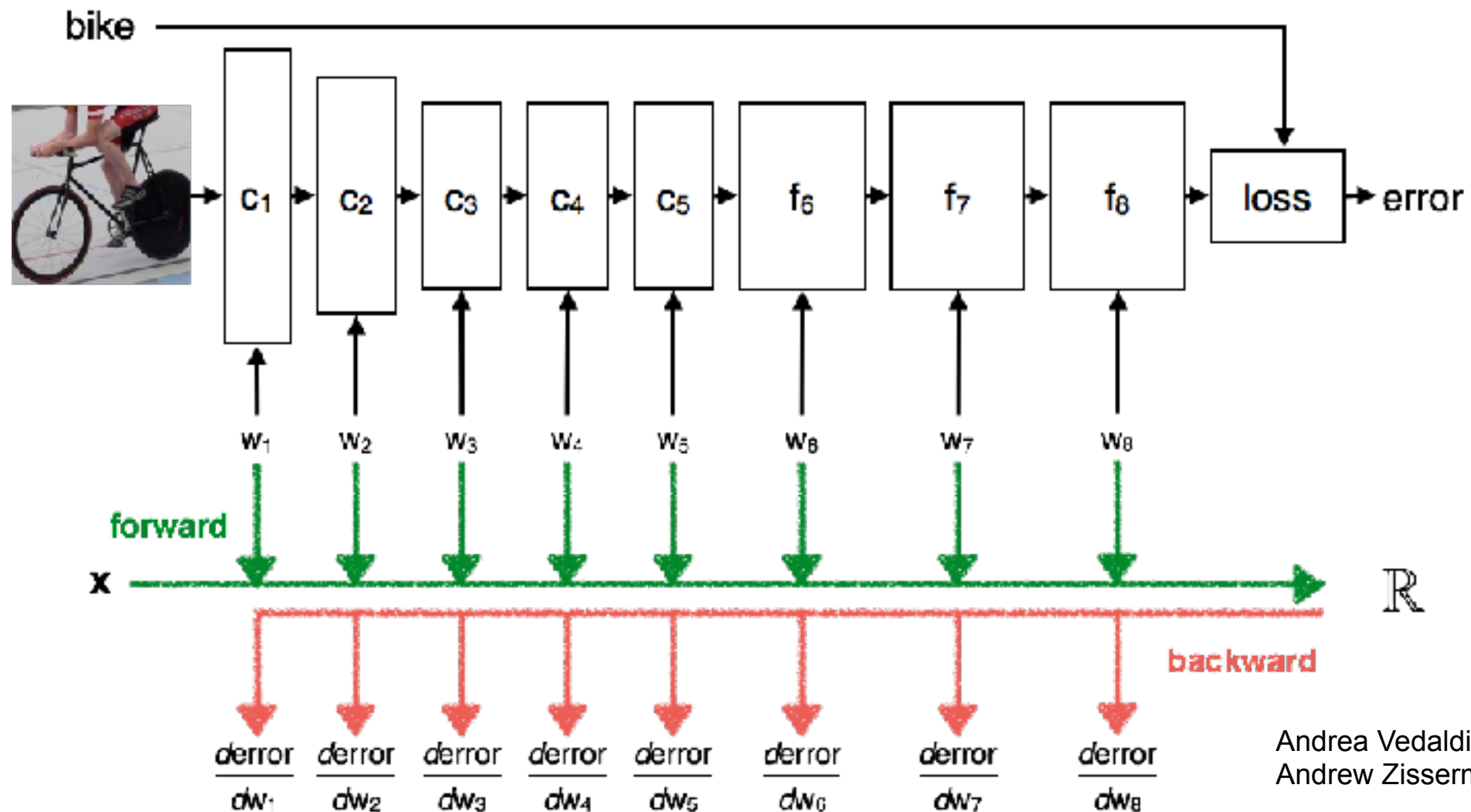
ReLU (rectified Linear Unit)



Deep Neural Network



Backward pass ...



Andrea Vedaldi and
Andrew Zisserman

Score, Loss, Optimization?

Deep Neural Network



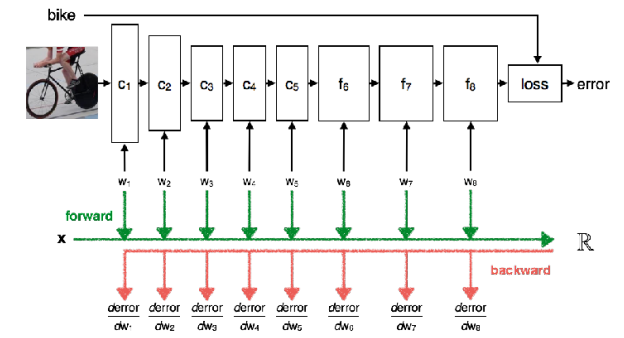
Optimisation/Training:

start with random weights & iteratively refine to get lower loss

```
while True:
    weights_grad = evaluate_gradient(loss_fun, data, weights)
    weights += - step_size * weights_grad # perform parameter update
```

- **miniB SGD**: gradient over batches. improve performance
- **backpropagation**: gradient analytically using *chain rule*
 - gates communicate to each other what they need in order to increase the final output (score).
 - allows to optimize relatively arbitrary loss functions (to define all kinds of NN, e.g. CNN)

Deep Neural Network



Optimisation/Training in practice:

- Network = operations *chained* together, each one has a simple derivative
- Graph structure. The nodes implement the **forward()** / **backward()** API

- **forward pass function** (local score from its *forward* input)
= **score** (using layer weights)



$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x)$$

$$f = q z$$

- **backward pass function** (local gradient from its *backwards* input)
= $d_error / d_layer\text{-}weights$

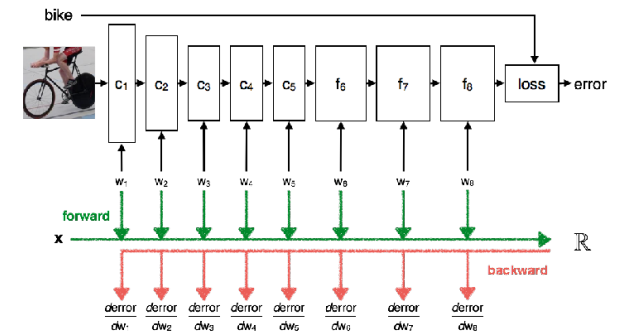


$$\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$$

$$df/dz = q$$

- **update** $W_{1,i} = W_{1,i} - lr \frac{\partial L}{\partial W_1} ; \dots$

Deep Neural Network



Optimisation/Training - Toy example of back propagation

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x)$$

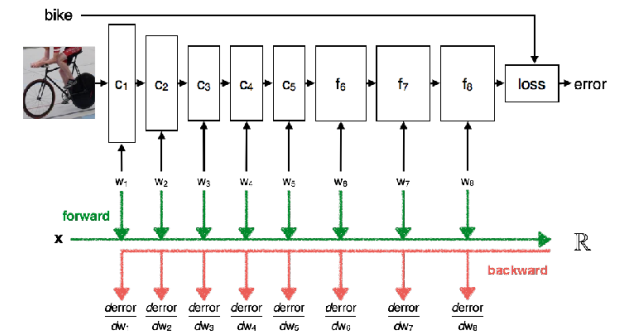
$$\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$$

How do we do this?

*Analytically would be just too long for “deep” sizes ...
L is not directly a function of all W, but is a function of a function of a function, etc*

—> CHAIN RULE

Deep Neural Network



Optimisation/Training - Toy example of back propagation



How do we do this?
Analytically would be just too long for “deep” sizes ...

$$f(x, y, z) = (x + y)z$$

Chain Rule

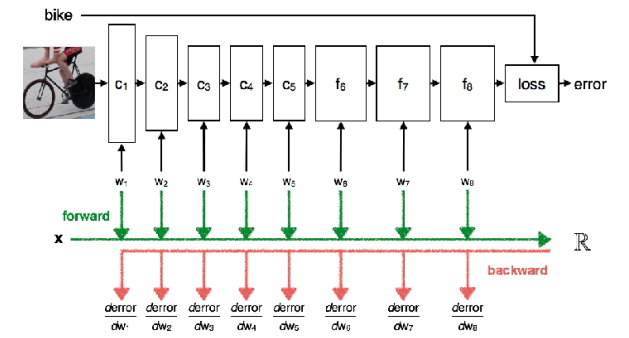
$$f = q \cdot z$$

$$df/dq = z$$

...

$$df/dx = df/dq * dq/dx$$

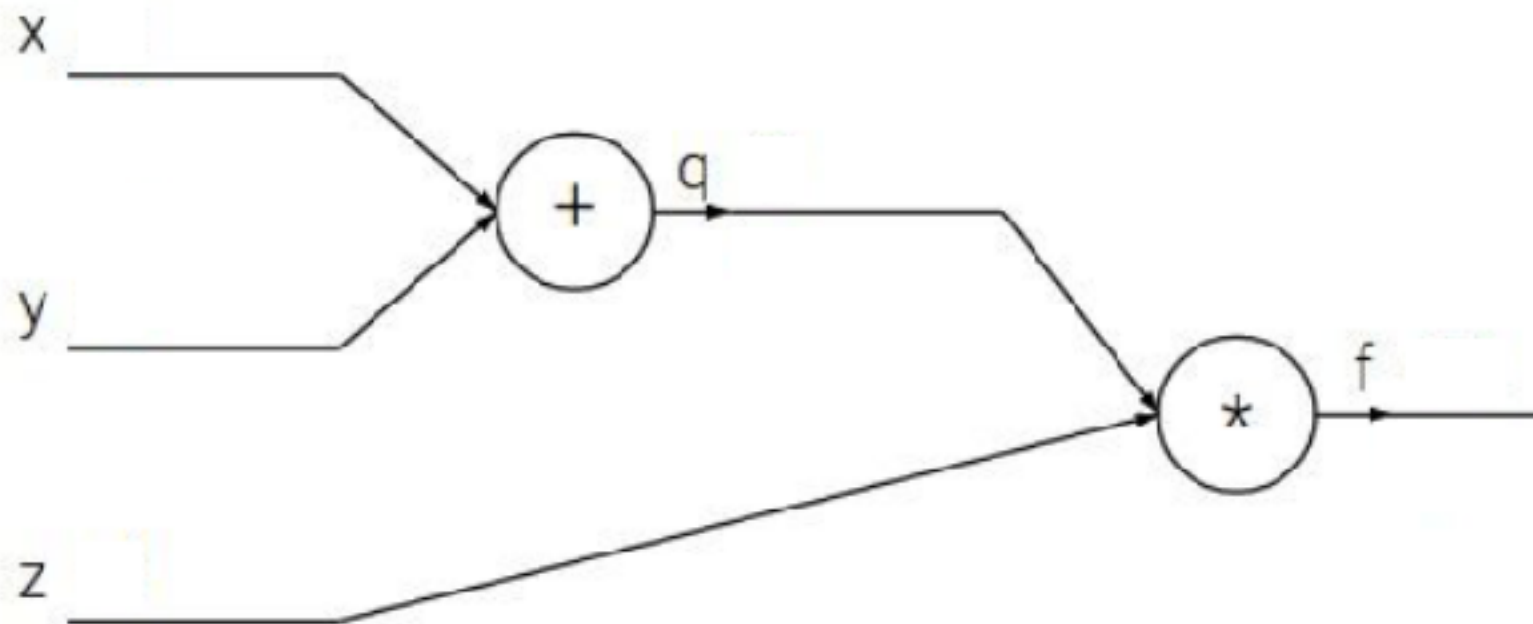
Deep Neural Network



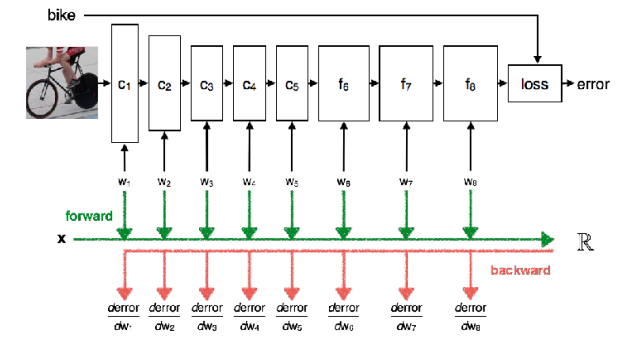
Optimisation/Training - Toy example of back propagation

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



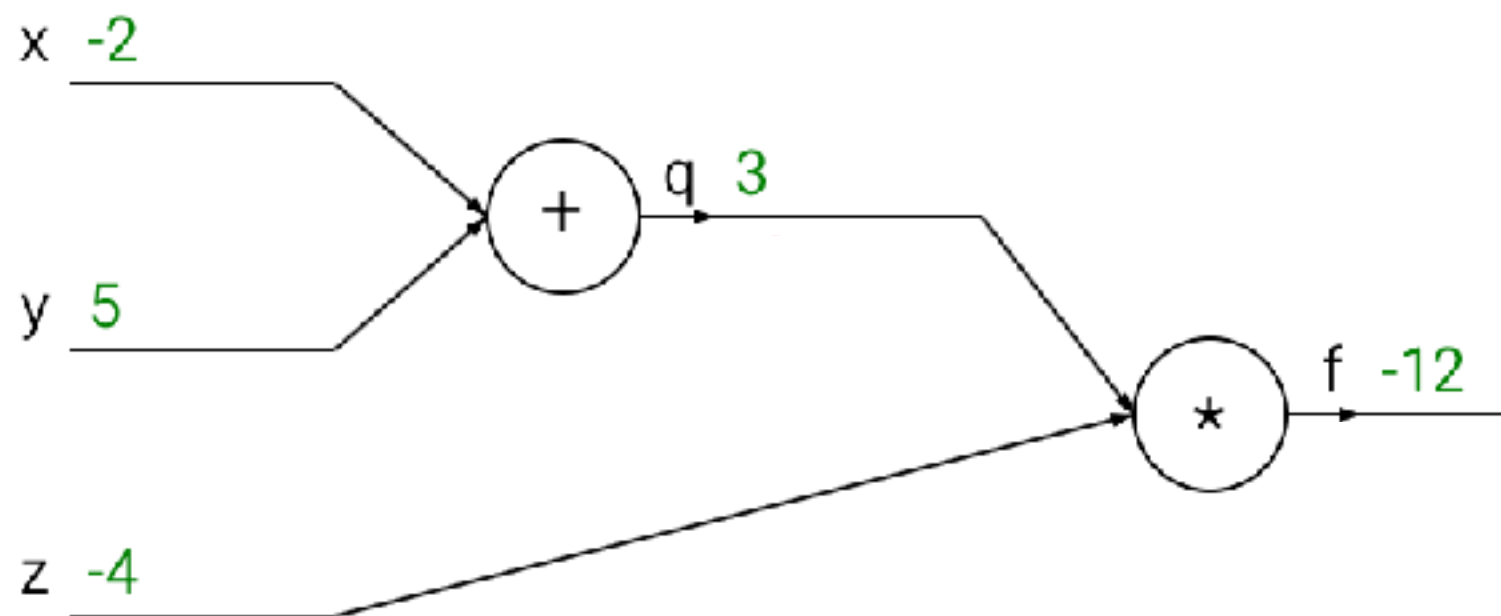
Deep Learning



Optimisation/Training - Toy example of back propagation

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

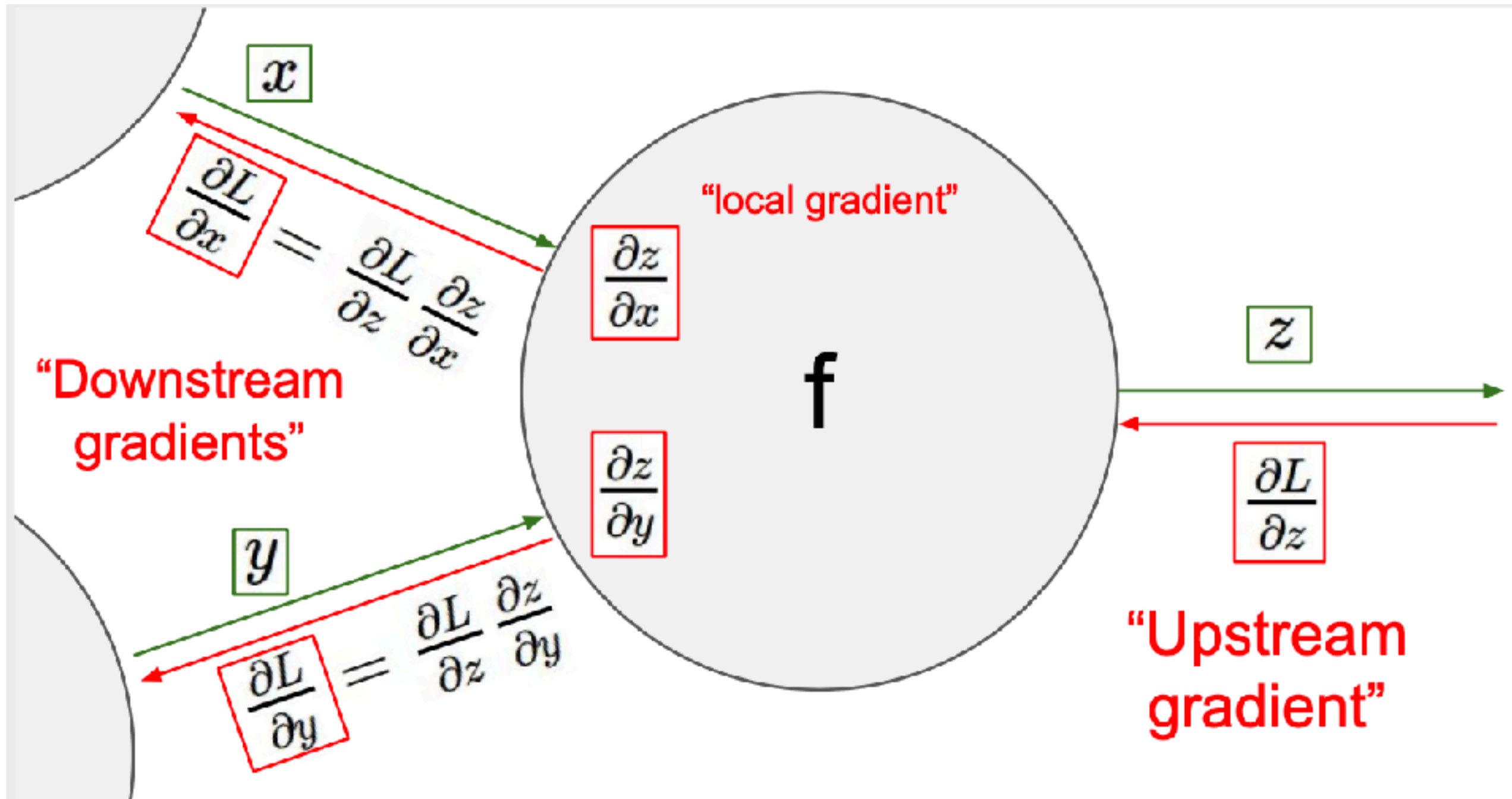
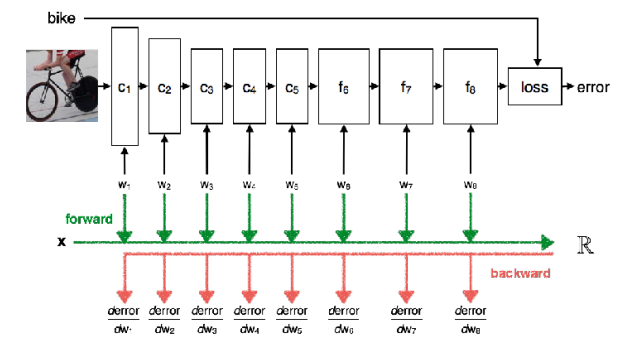
Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

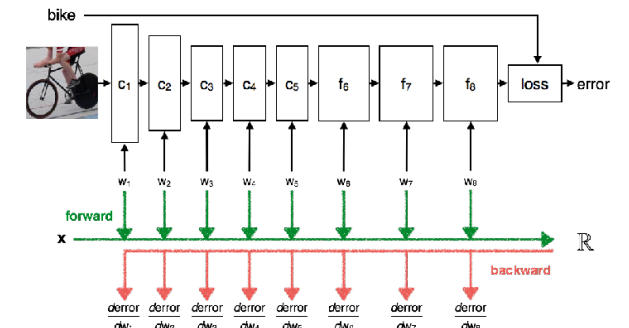
Upstream
gradient

Local
gradient

Deep Neural Network



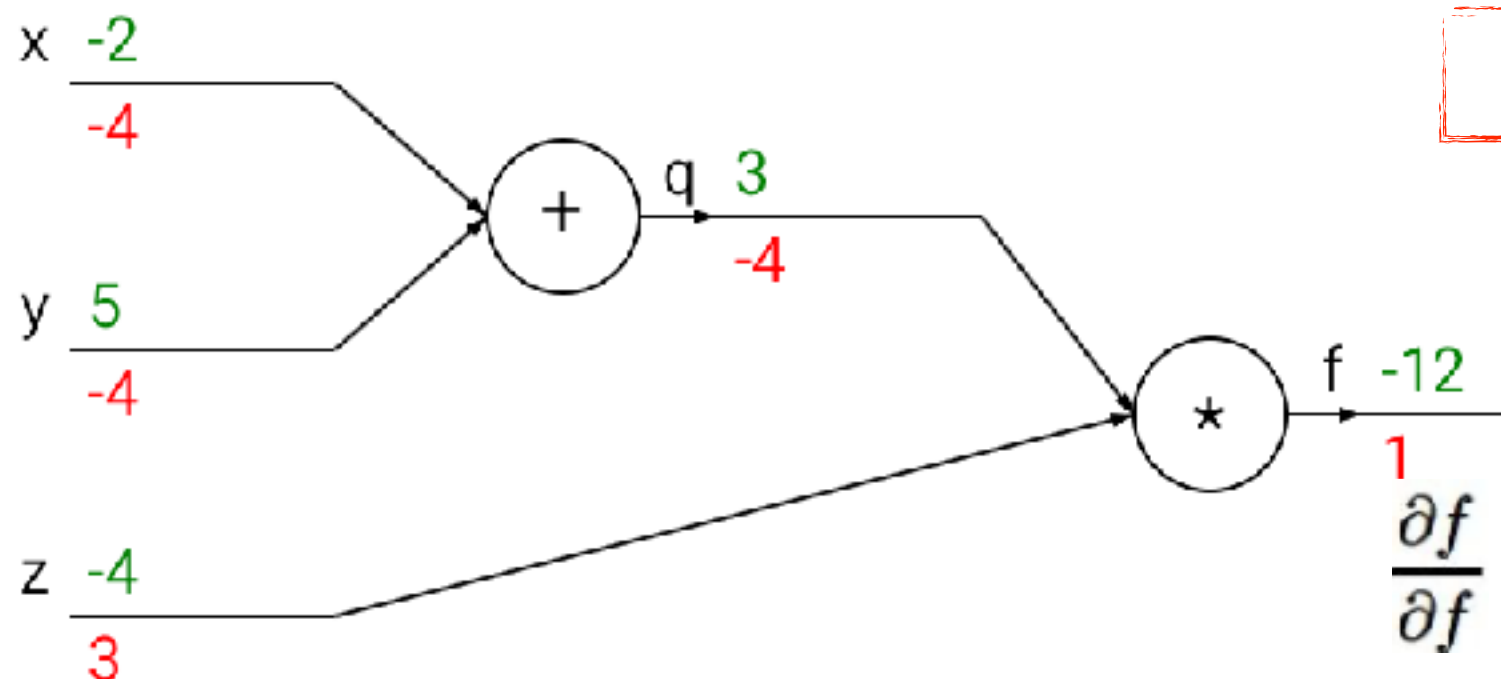
Deep Learning



Optimisation/Training - Toy example of back propagation

$$f(x, y, z) = (x + y)z$$

e.g. $x = -2$, $y = 5$, $z = -4$



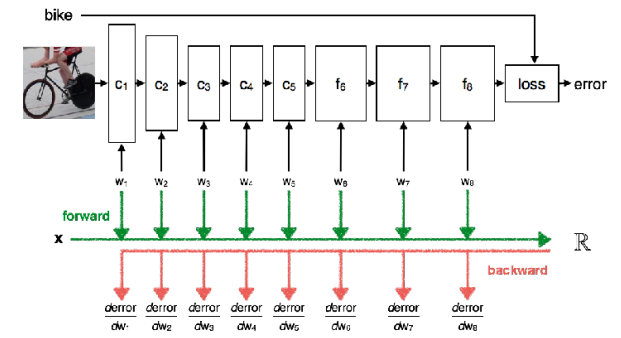
And what does this mean?

Weights update:

$$w = w - lr * dw$$

$$W_{1,i} = W_{1,i} - lr \frac{\partial L}{\partial W_{1,i}} ; \dots$$

Deep Neural Network



Optimisation/Training. Summary of simple common practices:

- **Normalize** input data
- **Initialize** weights. Not zero but small random numbers.
- **Batch normalisation**: forcing the activations throughout a network to take on a unit gaussian distribution at the beginning of the training
- Strong **regularization** - avoid overfitting (regularization strenght, dropout, augmentation, ...)
- Different **loss functions** depending on the task
- **Hyper-parameters** tuning

Bibliography - Resources for materials in this block

- Stanford online materials on Deep learning for Computer Vision (<http://cs231n.stanford.edu>) and Deep Learning (<https://cs230.stanford.edu/>)
- Ian Goodfellow, Yoshua Bengio, Aaron Courville, **Deep Learning**, MIT Press, 2016.
<http://www.deeplearningbook.org>